Partial Quantum Search LLM Evaluation

Failure analysis of quantum query-complexity reasoning across 120 model runs

Ustunpasa Igdir

June 2026

1. Summary

This project evaluates how language models handle a specialized quantum-computation reasoning task: calculating the query cost of a partial quantum search algorithm and comparing it with standard Grover search.

The task is mathematically compact, but it tests several reasoning layers at once:

- formula use;
- numerical substitution;
- quantum query-cost interpretation;
- parallel versus sequential local search;
- comparison against the correct Grover baseline;
- net saved-query calculation;
- preservation of correction terms in scaled numerical cases.

The main finding is:

> **LLMs may reproduce the correct formulas while still failing the operational cost model.**

The most frequent critical failures were not caused by complete formula ignorance. They were caused by incorrect interpretation of what the query count represents.

2. Evaluation target

The benchmark focuses on partial quantum search. The central issue is not whether a model can copy the formula, but whether it can apply the formula within the correct cost model.

In the base task, a database contains:

$$N = 10^6$$

items divided into:

$$K = 100$$

equally sized blocks. Exactly one item is marked.

The block size is:

$$b = \frac{N}{K}.$$

The partial quantum search algorithm performs:

$$\frac{\pi}{4}\sqrt{N} - \sqrt{\frac{3b}{4}}$$

global Grover iterations, followed by:

$$\frac{\pi}{6}\sqrt{b}$$

local Grover iterations in every block, performed in parallel.

One final global inversion about the average is also performed. In the base task, this final inversion is not counted as an oracle query.

The required outputs are:

- block size;
- global-stage query count;
- local-stage query count;
- total query count;
- query savings relative to standard Grover search.

3. Gold answer for the base task

The block size is:

$$b = \frac{10^6}{100} = 10,000.$$

Therefore:

$$\sqrt{N} = 1000$$

and:

$$\sqrt{b} = 100.$$

The global-stage query count is:

$$Q_{global} = \frac{\pi}{4}\sqrt{N} - \sqrt{\frac{3b}{4}}.$$

Substitution gives:

$$Q_{global} = \frac{\pi}{4}(1000) - \sqrt{\frac{3(10,000)}{4}}.$$

$$Q_{global} = 785.40 - 86.60 = 698.80.$$

The local-stage query count is:

$$Q_{local} = \frac{\pi}{6}\sqrt{b}.$$

$$Q_{local} = \frac{\pi}{6}(100) = 52.36.$$

Because the local searches are performed in parallel, this value is **not** multiplied by K.

The total query count is:

$$Q_{total} = Q_{global} + Q_{local}.$$

$$Q_{total} = 698.80 + 52.36 = 751.16.$$

Standard Grover search requires:

$$Q_{Grover} = \frac{\pi}{4}\sqrt{N} = 785.40.$$

The saved-query count is:

$$Q_{saved} = Q_{Grover} - Q_{total}.$$

$$Q_{saved} = 785.40 - 751.16 = 34.24.$$

Quantity	Gold value
Block size	10,000
Global-stage queries	698.80
Local-stage queries	52.36
Total queries	751.16
Standard Grover queries	785.40
Queries saved	34.24

4. Dataset summary

The evaluation used 120 model runs.

Category	Count	Interpretation
Total runs	120	All attempted generations
No-result / discarded runs	8	Outputs that did not produce usable results
Successful outputs	112	Outputs available for evaluation
Failed reviewed outputs	58	Outputs that failed review
Critical failures	19	Failures with serious conceptual consequences

From the 112 successful outputs:

$$\frac{58}{112} = 51.79\%$$

failed review.

$$\frac{19}{112} = 16.96\%$$

were critical failures.

Among the 58 reviewed failures:

$$\frac{19}{58} = 32.76\%$$

were critical.

The failure density is high enough to support meaningful analysis. The benchmark did not only detect formatting problems or minor arithmetic variation; it exposed repeated conceptual failure patterns.

5. Model capability tiers

The failing models were grouped into practical capability tiers for interpretation. This is not a universal model ranking. It is a project-specific classification used to compare failure types across model classes.

Tier	Models	Practical description
Advanced / stronger hosted model	Mistral Medium 3.5	Expected to handle multi-step reasoning more reliably
Mid-size open model	Gemma 3 12B IT	Capable open instruction model, but not frontier-class
Compact / local / edge-oriented models	Qwen3 8B, Llama 3.1 8B Instruct, Ministral 3 8B	Smaller models where reasoning failures are more expected, but still diagnostically useful

This classification helped separate expected compact-model weaknesses from more concerning failures in stronger models.

Compact models mostly failed through cost aggregation errors. Larger or stronger models showed more subtle failures involving magnitude separation and correction-term preservation.

6. Critical failure distribution

Critical labels can overlap. One response may have more than one critical label.

Critical label	Count among 19 critical failures	Meaning
`C_FALSE_SAVING_CLAIM`	16	Wrong conclusion about query savings
`C_PARALLEL_AS_SEQUENTIAL`	12	Treats parallel local searches as sequential

Critical label	Count among 19 critical failures	Meaning
C_CORRECTION_TERM_DROPPED	4	Drops a correction term needed for saved-query calculation
C_WRONG_GROVER_BASELINE	3	Uses the wrong standard Grover comparison baseline

The dominant critical failure was a wrong conclusion about query savings. This matters because the final scientific interpretation was often wrong even when intermediate formulas were partly correct.

7. Main finding

The central finding is:

Formula-level correctness does not guarantee cost-model correctness.

Many failed responses correctly calculated some or all of these values:

$$b = 10,000$$

$$\sqrt{N} = 1000$$

$$\sqrt{b} = 100$$

$$Q_{global} = 698.80$$

$$Q_{local} = 52.36$$

$$Q_{Grover} = 785.40$$

Yet those same responses could still fail by misunderstanding what the query cost meant.

The most common serious error was:

Correct:

$$Q_{local} = 52.36$$

Incorrect:

$$Q_{local,total} = 100 \times 52.36 = 5236.00.$$

This single error changes the total from:

$$751.16$$

to:

$$5934.80.$$

The resulting false conclusion is that partial search is worse than standard Grover search. The correct conclusion is that the algorithm saves:

$$34.24$$

queries.

8. Error taxonomy

The review used a label-based error taxonomy.

Label	Meaning	Severity
`C_FALSE_SAVING_CLAIM`	Wrong final conclusion about saved queries	Critical
`C_PARALLEL_AS_SEQUENTIAL`	Multiplies parallel local-search cost by K	Critical
`C_CORRECTION_TERM_DROPPED`	Drops a correction term needed for saved-query count	Critical
`C_WRONG_GROVER_BASELINE`	Uses the wrong standard Grover baseline	Critical
`C_UNSUPPORTED_ALTERNATIVE_BASELINE`	Invents a comparison baseline not requested by the prompt	Serious
`R_CONTRADICTORY_RESPONSE`	States the correct rule but violates it in the calculation	Serious / diagnostic
`R_REPETITIVE_LOOP`	Loops or repeats instead of resolving the task	Diagnostic
`N_TOTAL_VALUE_ERROR`	Wrong total query count	Numeric
`N_SAVED_VALUE_ERROR`	Wrong saved-query count	Numeric
`N_PRECISION_INSUFFICIENT`	Not enough precision for the target quantity	Numeric / presentation
`E_EXTRACTION_MISS`	Parser missed a value that appears present in the response	Evaluation-pipeline issue

The `E_EXTRACTION_MISS` label separates evaluation-pipeline extraction misses from actual model-side reasoning failures.

9. Failure type 1: false saving claim

The most frequent critical label was `C_FALSE_SAVING_CLAIM`.

This label applies when the model gives the wrong conclusion about the algorithm's advantage.

The correct saved-query logic is:

$$Q_{saved} = Q_{Grover} - Q_{partial}.$$

For the base problem:

$$Q_{saved} = 785.40 - 751.16 = 34.24.$$

Failed models often reported a negative saving, zero saving, or a saving computed against the wrong baseline.

This is critical because the saved-query comparison is the scientific point of the problem. A response can contain correct intermediate calculations but still fail the task if it says the algorithm does not save queries when it actually does.

10. Failure type 2: parallel-as-sequential

The clearest repeated conceptual failure was `C_PARALLEL_AS_SEQUENTIAL`.

The problem states that local searches are performed in parallel. Therefore:

$$Q_{local} = \frac{\pi}{6}\sqrt{b}.$$

It is not:

$$K\frac{\pi}{6}\sqrt{b}.$$

Many failed responses stated the parallel condition correctly in prose, then multiplied the local query count by K anyway.

This pattern is important because it shows that a model may recognize a constraint linguistically without applying it operationally.

11. Failure type 3: correction term dropped

The scale variant exposed a different failure mode: `C_CORRECTION_TERM_DROPPED`.

In the scaled problem, the leading term is of order:

$$10^{49}$$

while the correction terms are of order:

$$10^{39}.$$

A term of order 10^{39} is small relative to 10^{49}, but it is not irrelevant. The saved-query count itself is determined by correction-scale terms.

Some models rounded:

$$7.853981633 \times 10^{49}$$

to:

$$7.854 \times 10^{49}$$

and lost the correction term entirely.

This made:

$$Q_{total} \approx Q_{Grover}$$

and led to the false conclusion that the algorithm saved zero queries.

The relevant principle is:

> A term can be small relative to the leading term and still be decisive for the question being asked.

12. Failure type 4: wrong Grover baseline

The correct standard Grover baseline is:

$$Q_{Grover} = \frac{\pi}{4}\sqrt{N}.$$

For the base problem:

$$Q_{Grover} = 785.40.$$

Some models used the wrong baseline. This happened in two main ways:

1. miscomputing $\sqrt{10^6}$;
2. inventing a different comparison baseline after a first calculation produced negative savings.

The second case is a reasoning-drift problem. The model detected that something was wrong, but repaired the wrong part of the solution. Instead of correcting the local-cost aggregation error, it changed the comparison target.

13. Failure type 5: unsupported alternative baseline

Some models compared the partial-search algorithm against a baseline that was not requested, such as independent Grover searches over all blocks.

The prompt asked for comparison against standard Grover search over the full database:

$$\frac{\pi}{4}\sqrt{N}.$$

Changing the baseline changes the problem.

This failure matters because a model may try to rescue a wrong answer by silently modifying the task. In scientific evaluation, that is a serious reliability issue.

14. Contradictory responses

A repeated pattern was:

1. the model states that local searches are performed in parallel;
2. the model states that the local cost is not multiplied by K;
3. the model then multiplies the local cost by K.

This is not only an arithmetic mistake. It is an internal contradiction.

For LLM evaluation, this matters because fluent explanations can hide incompatible reasoning steps. A response can sound correct while failing to obey its own stated rule.

15. Self-correction failures

Some models noticed that their answer gave negative saved queries.

That should have triggered the correct diagnostic question:

> Was the parallel local-search cost accidentally multiplied by K?

Several responses did not identify that cause. Instead, they repeated the calculation, looped, or introduced a different unsupported correction.

This shows that model self-correction is not automatically reliable. A model may detect that a result is suspicious while still failing to locate the source of the error.

A useful future label is `R_FAILED_SELF_CORRECTION`, which would capture cases where a model recognizes inconsistency but repairs the wrong part of the reasoning.

16. Scenario-level findings

16.1 `PQS_BASE_001`

This was the main conceptual test. It tested whether the model understood that local searches were parallel.

Main failure:

$$Q_{local} \to KQ_{local}$$

Assessment: `PQS_BASE_001` is effective for testing operational query-cost reasoning.

16.2 `PQS_SCALE_001`

This variant tested scientific notation and correction-term preservation.

Main failure:

$$Q_{global} = 7.85398 \times 10^{49} - 8.66025 \times 10^{39}$$

being reported as simply:

$$7.854 \times 10^{49}.$$

Assessment: `PQS_SCALE_001` is highly valuable because it tests whether the model preserves small relative corrections that determine the final answer.

16.3 `PQS_COUNT_001`

This variant tested whether the model could adapt when the query-counting convention changed.

Assessment: `PQS_COUNT_001` is useful for detecting models that reuse the base answer without adjusting to the prompt. Its prompt should state the counting convention very explicitly.

16.4 `PQS_VAR_SEQ_001`

This variant tested a sequential local-search version.

Assessment: `PQS_VAR_SEQ_001` is useful as a control case, because multiplying by K may be appropriate in a sequential variant.

17. Model-level findings

17.1 Mistral Medium 3.5

Tier: advanced / stronger hosted model.

Observed weakness: scale-sensitive arithmetic and correction-term handling.

The notable issue was not complete formula failure. The model often handled formula structure correctly. The failure was magnitude handling: in a scale-sensitive case, terms of very different orders were combined incorrectly, producing an impossible conclusion where partial search appeared worse than Grover search.

Interpretation: stronger models can still fail when the decisive result is a small difference between large quantities.

17.2 Gemma 3 12B IT

Tier: mid-size open model.

Observed weakness: correction-term dropping through rounding.

The model could write the formula correctly, but rounded away the term that determined the saved-query count.

Interpretation: formula recall did not guarantee scale-sensitive numerical discipline.

17.3 Qwen3 8B

Tier: compact open model.

Observed weakness: repeated parallel-as-sequential failure.

The model often computed the local cost per block correctly, then multiplied it by K even though the searches were parallel.

Interpretation: the model recognized parts of the prompt but failed to apply the parallel-cost constraint.

17.4 Llama 3.1 8B Instruct

Tier: compact open instruction model.

Observed weaknesses: parallel/sequential confusion, arithmetic errors, contradiction, and looping self-correction.

Interpretation: the model was unstable under multi-step query accounting. It sometimes noticed that the answer was suspicious but failed to repair the correct step.

17.5 Ministral 3 8B

Tier: compact / edge-oriented model.

Observed weaknesses: parallel-as-sequential failure and unsupported alternative baselines.

Interpretation: when the first calculation led to an implausible result, the model sometimes changed the baseline instead of correcting the local-cost aggregation error.

18. What the benchmark tests

This benchmark tests multiple skills simultaneously.

18.1 Formula recall

Can the model use:

$$\frac{\pi}{4}\sqrt{N} - \sqrt{\frac{3b}{4}}$$

and:

$$\frac{\pi}{6}\sqrt{b}?$$

18.2 Numerical substitution

Can the model compute:

$$b = 10,000, \quad \sqrt{N} = 1000, \quad \sqrt{b} = 100?$$

18.3 Query model understanding

Does the model understand what counts as an oracle query? Does it avoid adding extra oracle costs that are not specified?

18.4 Parallelism

Does the model understand that parallel local searches contribute query depth, not sequential total work?

18.5 Baseline comparison

Does the model compare against:

$$\frac{\pi}{4}\sqrt{N}$$

instead of inventing another baseline?

18.6 Net saving

Does the model compute:

$$Q_{Grover} - Q_{partial}$$

rather than merely reporting the omitted global correction?

18.7 Precision discipline

Does the model preserve correction terms when they are small relative to the leading term but decisive for the saved-query count?

This combination makes the task a structured reasoning test rather than a simple calculator test.

19. Label hygiene and evaluation-pipeline cleanup

Some numeric error labels were caused by extraction limitations rather than model reasoning failures.

The autograder sometimes recorded `observed None` for values such as b, $\sqrt{N}$, or $\sqrt{b}$, even when the human review note indicated that the model had computed them correctly.

The final taxonomy therefore includes `E_EXTRACTION_MISS`.

This label means:

> The value appears to be present in the model response, but the evaluator or parser failed to extract it.

This distinction separates:

1. model reasoning failure;
2. model presentation failure;
3. evaluator extraction failure.

Without this distinction, the analysis would overstate some numeric mistakes.

20. Conclusions

20.1 Main conclusion

LLMs may reproduce quantum-search formulas while failing the query-complexity interpretation.

20.2 Detailed conclusions

1. The benchmark distinguishes formula recall from operational reasoning.
2. The most common serious failure was not using the wrong formula; it was using the formula under the wrong cost interpretation.
3. The most damaging error was treating parallel local Grover searches as sequential.
4. Several models stated the parallelism condition but failed to apply it.
5. The final saved-query count was the most fragile output.
6. A wrong local-stage query count usually caused a wrong total and then a wrong scientific conclusion.
7. False saving claims are critical because they reverse the meaning of the algorithm.
8. Large-scale variants exposed a different weakness: models dropped correction terms that were small relative to the leading term but decisive for the answer.
9. Rounding can become a conceptual failure when the target is a small difference between large quantities.
10. Some models used unsupported baselines when their initial answer looked inconsistent.
11. Model self-correction was unreliable in several cases.
12. Compact models mostly failed through cost aggregation errors.
13. Larger or stronger models showed more subtle scale-sensitive failures.
14. Different variants exposed different failure modes.
15. Separating extraction misses from model errors improved the reliability of the analysis.

21. Limitations and future work

21.1 Additional controlled variants

Future versions should add:

1. a variant where local searches are explicitly sequential;

2. a variant where the final inversion is counted as one query;
3. a variant where the prompt asks separately for oracle queries and total operations;
4. a variant where the model must explain why the local cost is not multiplied by K;
5. a scale variant where the saved-query term must be reported separately.

21.2 Structured output requirements

Future prompts should require a final table with these fields:

- `b`
- `sqrt_N`
- `sqrt_b`
- `global_queries`
- `local_queries`
- `total_queries`
- `grover_queries`
- `saved_queries`
- `parallel_cost_multiplied_by_K: yes/no`

This would reduce extraction misses.

21.3 Direct self-check

A useful self-check field would be:

> Did the solution multiply the local-stage query count by K? Explain why or why not.

This directly targets the most common critical failure.

21.4 Confidence versus correctness

Future analysis should record whether the model sounded confident while wrong.

Polished wrong answers are more dangerous than visibly uncertain wrong answers.

22. Evidence layer

The cleaned failure log is retained as the technical evidence layer for this report. It contains the inspected failed responses, final verdicts, critical labels, numeric mismatch notes, reasoning inconsistency notes, and evaluator comments.

The public report summarizes the patterns. The cleaned failure log preserves the review trail.

23. Final assessment

This evaluation shows that partial quantum search is a useful targeted benchmark for STEM-oriented LLM evaluation.

The benchmark is especially effective because it detects a specific and consequential reasoning gap:

Formula recognition is not the same as cost-model understanding.

The strongest result is that models can reproduce the correct quantum-search formulas while failing the operational interpretation of query complexity. The cleaned failure analysis supports this conclusion across model tiers and scenario variants.